

JAVA

__/__/

Introduction

{Purely Object-Oriented}

→ JAVA is a high-level, Object-oriented programming developed by Sun Microsystems in 1995.

→ It is mostly used for building applications

→ Web applications

→ Android apps

→ & Enterprise systems.

• Java is guaranteed to be "Write Once, Run anywhere."

• Java is → Object oriented, platform Independent

→ Simple & Secure

→ Architecture-neutral

→ Portable

→ Robust

→ Interpreted

→ High performance

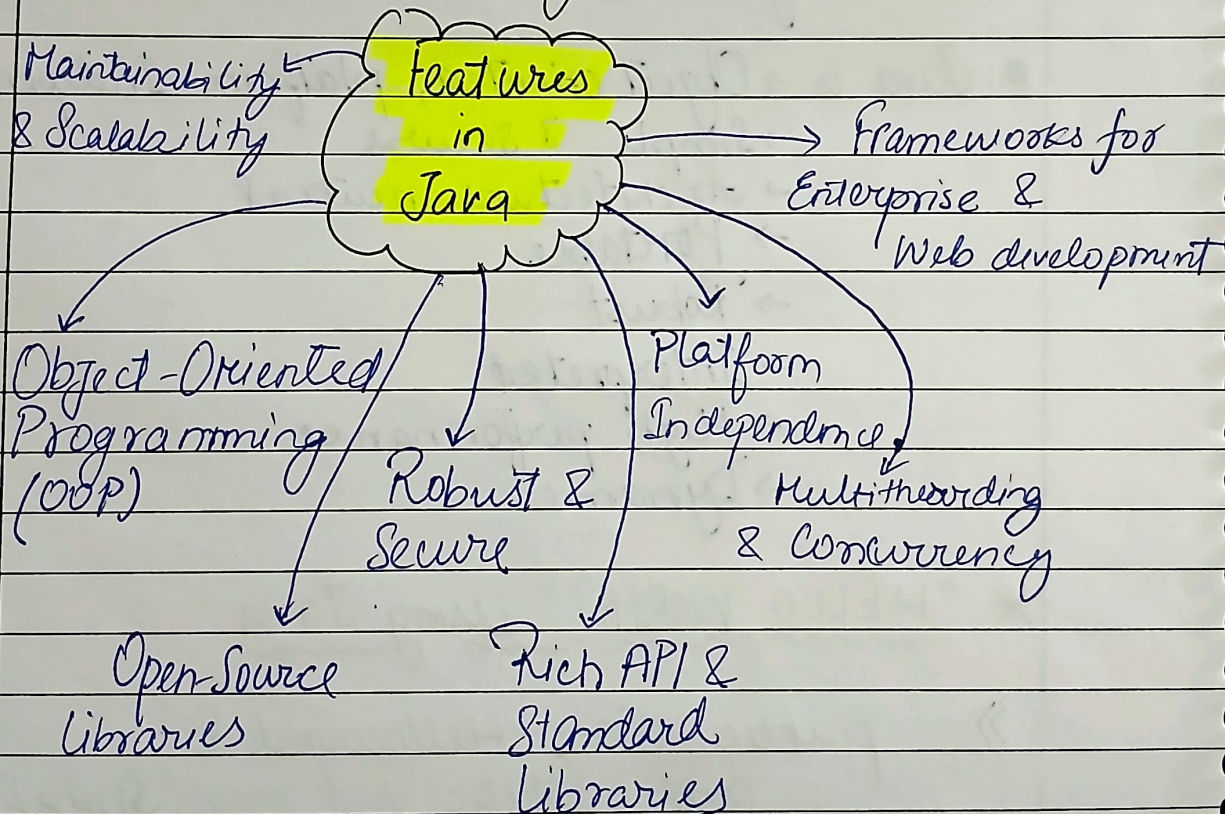
→ Dynamic.

* "HELLO WORLD!" Using Java

```
» public class Helloworld {  
    public static void main(String[] args){  
        System.out.println("Hello World!");  
    }  
}
```

• Explanation

- Public class HelloWorld ⇒ Define the class named HelloWorld.
- public Static void main(String[] args) ⇒ The entry point of every Java program.
- It tells the "JVM" where to start executing the program.
- System.out.println("Hello World")
 - Print the message to the console.



- Object-Oriented programming

→ Java supports OOP concepts to create modular & reusable code.

- Platform Independence

→ Java programs can run on any OS with a JVM.
JVM ⇒ Java Virtual Machine.

- Robust & Secure

→ Java ensures reliability and security through strong memory management & exception handling.

- Multithreading and Concurrency

→ Java allows concurrent execution of multiple tasks for efficiency.

- Rich API & Standard Libraries

→ Java provides extensive built-in libraries for various programming needs.

- Frameworks for Enterprise & Web Development

→ Java supports frameworks that simplify enterprise and web development application development.

• Open-Source Libraries

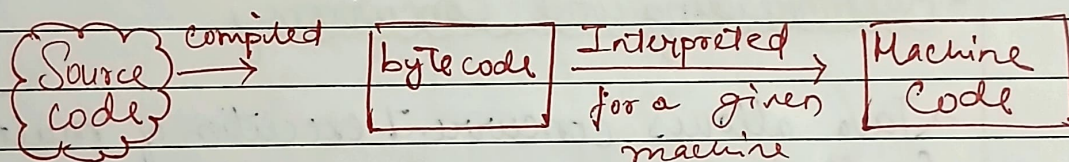
→ Java has a wide range of libraries to extend functionality and speed up development.

• Maintainability & Scalability

→ Java's structured design allow easy maintenance & growth of applications.

* How JAVA Works?

→ Java is compiled into the byte code & then it is interpreted to machine code.



* JDK (Java development Kit)

→ Collection of tools used for developing and running java programs.

* JRE (Java Runtime Environment)

→ Helps in executing programs developed in JAVA

* Naming conventions in Java

① Classes → For classes names should be nouns, in mixed case with the first letter of each interval word capitalized. (also known as PascalCase)

For eg:- MyClass, Nitin, MyTime, SchoolName.

② Functions & Variables → Variable (instance & local)

→ Names are noun & uses lower Camel Case

→ The first letter is lower case, & the first letter of subsequent word is capitalized.

Eg: userName, totalAmount, myName, main. etc.

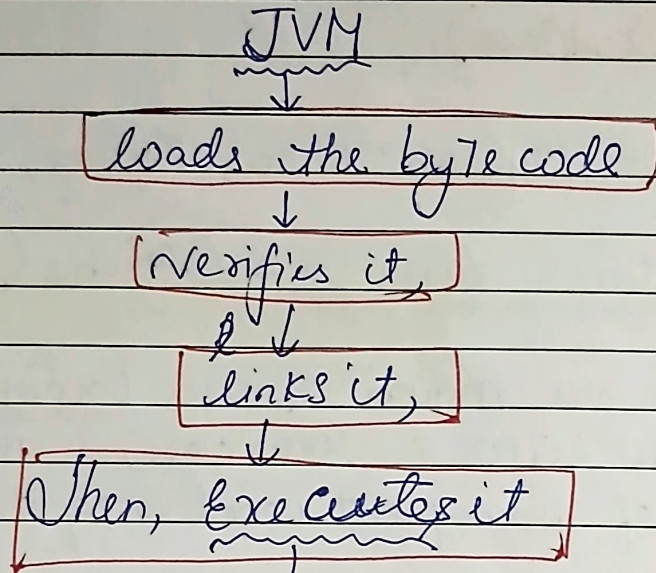
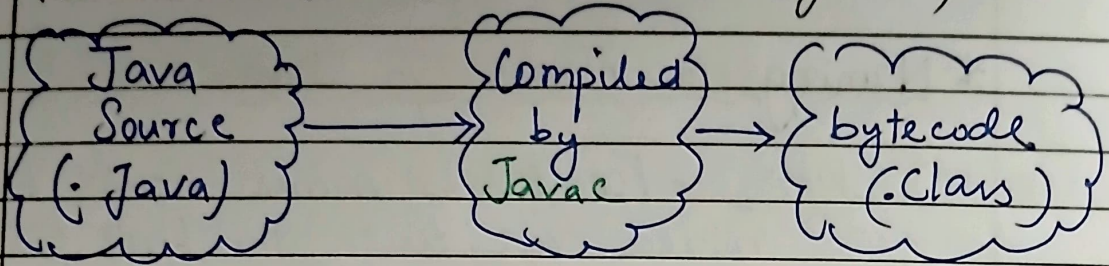
Java JVM, JRE & JDK

● JVM (Java Virtual Machine)

→ The JVM is a core component of the Java Runtime Environment (JRE) that allows Java programs to run on any platforms without modifications.

→ JVM acts as a interpreter between Java, bytecode & the underlying hardware, providing Java's famous Write once, Run Anywhere capability.

definition JVM → JVM is engine that runs JAVA programs.
→ It converts Java byte code (compiled code) into machine code (Understandable by the OS) - / - /



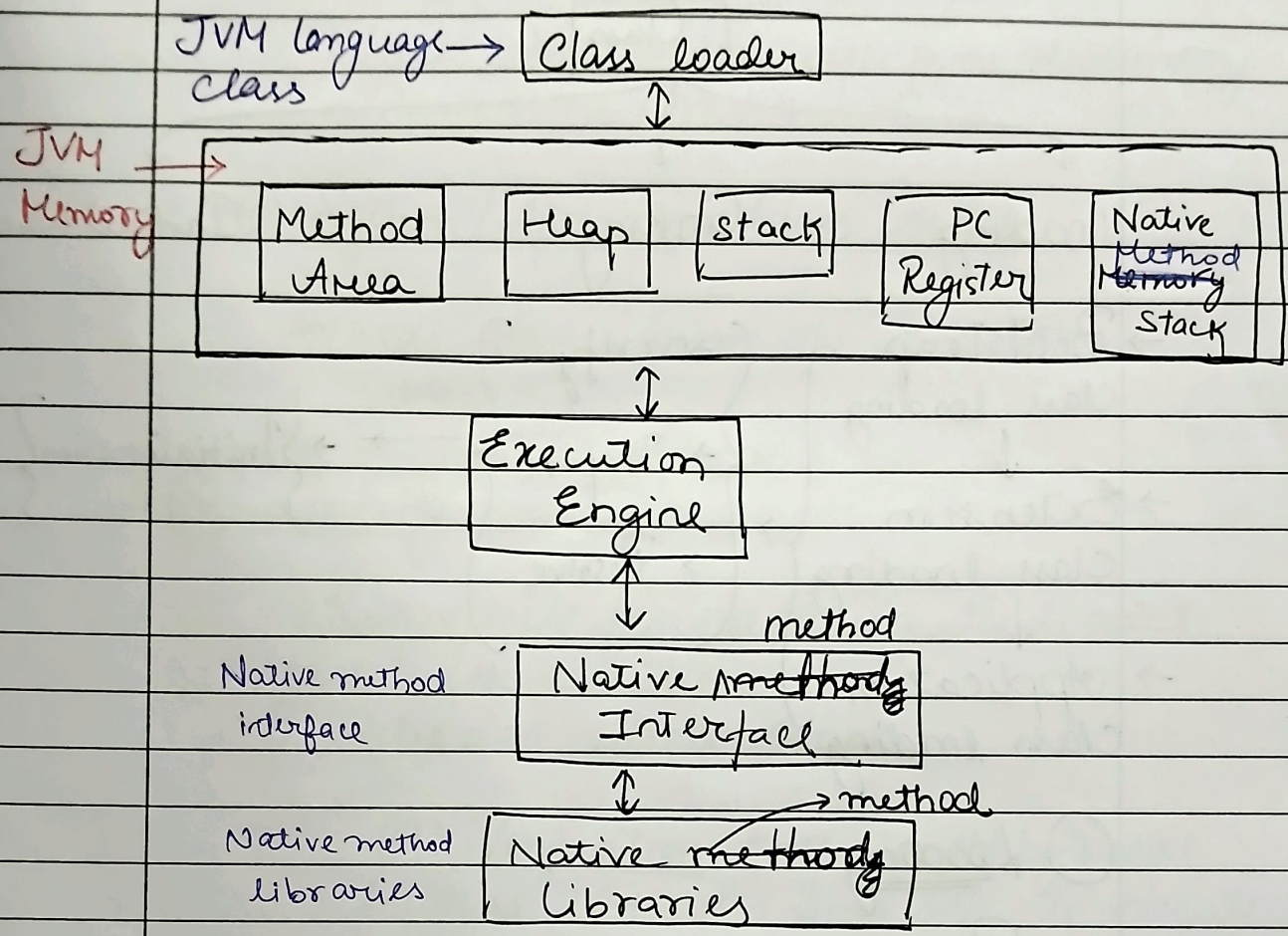
Execution involves

- interpreting bytecode
- or • Using Just-In-Time (JIT) Compilation to convert "hot" code into native machine code for performance

→ Garbage collection runs in background to reclaim memory from unused objects.

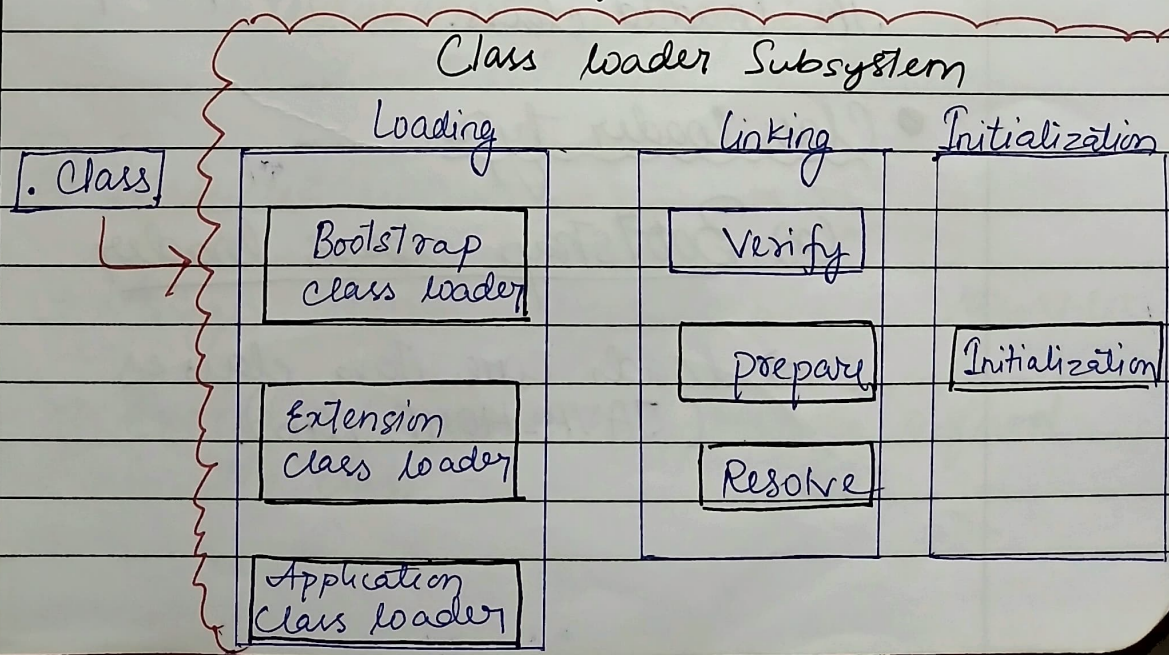
{ JVM consists of → Java Hotspot client & Server Virtual Machine }

* Architecture of JVM

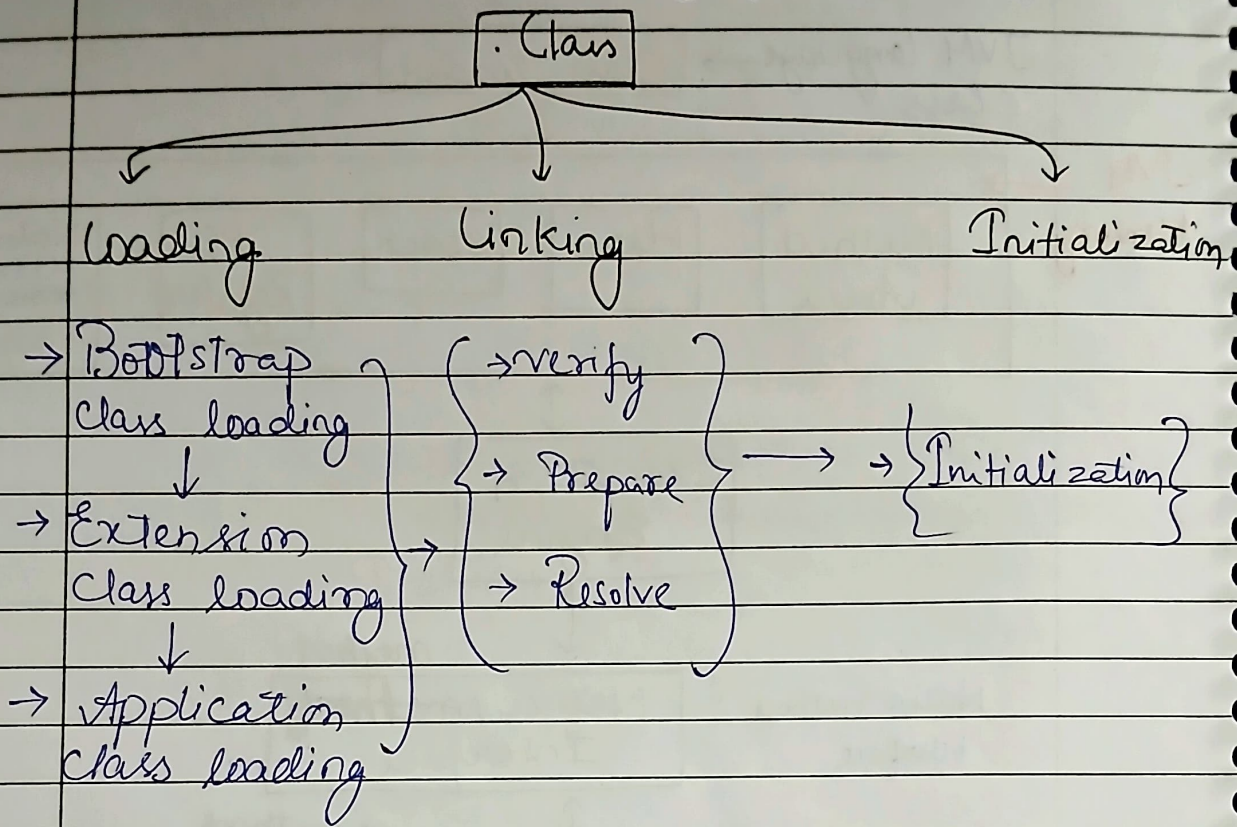


* Components of JVM architecture

1.) Class loader subsystem



Class loader Subsystem



① Loading

- Reads class files & stores class metadata in the Method area
- Creates a Class in the heap representing the loaded class.

• Class loader types

(a) Bootstrap class loader

- loads core Java classes (JAVA_HOME / Lib)

_ / _ / _

(b) Extension Class loader

→ loads classes from extensions dictionary (JAVA_HOME / jre / lib / ext).

(c) System / Application Class Loader

→ Loads classes from the application classpath

(2) Linking

→ Responsible for preparing the loaded class for execution.

→ It includes three steps:-

(a) Verification: Ensures the bytecode follows JVM rules & is safe to execute.

(b) Preparation: Allocate memory for static variables & assign default values.

(c) Resolution: Converts symbolic references into direct reference in memory.

(3) Initialization

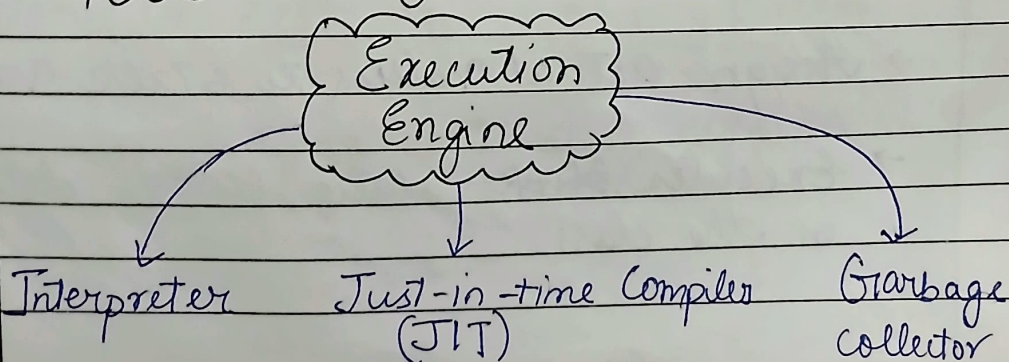
→ Assign actual values to static variables.

→ Executes blocks static blocks defined in the class.

* JVM memory areas

- Method areas → Stores class-level information like class name, parent class, methods, variables & static data. Shared across JVM.
- Heap areas → Stores all objects. Shared across JVM.
- ~~Stack areas~~ → Hold the address of the currently executing instruction for each thread.
- Stack → Each threads has its own runtime stack; stores method calls, local variables in stack frames. Destroyed when the thread ends.
- PC Registers → Hold the addresses of the currently executing instruction for each thread.
- Native Method Stack → Each thread has a spe separate stack for native method execution.

Execution Engine



- Execution engine executes the ".class" (bytecode). It reads the byte-code line by line, uses data and information present in various memory area & executes instructions. It can be classified into three parts.

① Interpreter :- It interprets the bytecode line by line and then executes. The disadvantages here is that when one method is called multiple times, every time interpretation is required.

② Just-in-time Compiler (JIT) → It is used to increase the efficiency of an interpreter. It compiles the entire bytecode and changes it to native code so whenever the interpreter sees repeated method calls. JIT provides direct native code for that part so re-interpretation is not required, thus efficiency is improved.

③ Garbage Collector → It destroys un-referenced objects.

* JAVA Method Interface

* JAVA Native Interface (JNI) or Native Method Interface

→ It is a interface that interacts with the Native Method Libraries & provides the native libraries (C, C++) required for the execution. It enables JVM to call C/C++ libraries and to be called by C/C++ libraries which may be specific to hardware.

5) Native Method Libraries

These are collection of native libraries required for executing native methods. They includes libraries written in languages like C & C++.

Java Runtime Environment (JRE)

- It is an open-access software distribution that includes a Java class library, specific tools, and a separate JVM.
- In Java, JRE is one of the main component of the Java Development Kit (JDK).

- JRE provides the environment required to run Java program.
- Java source code is first compiled into Java byte code.

↓
To run this byte code on any platform, we need the JRE.

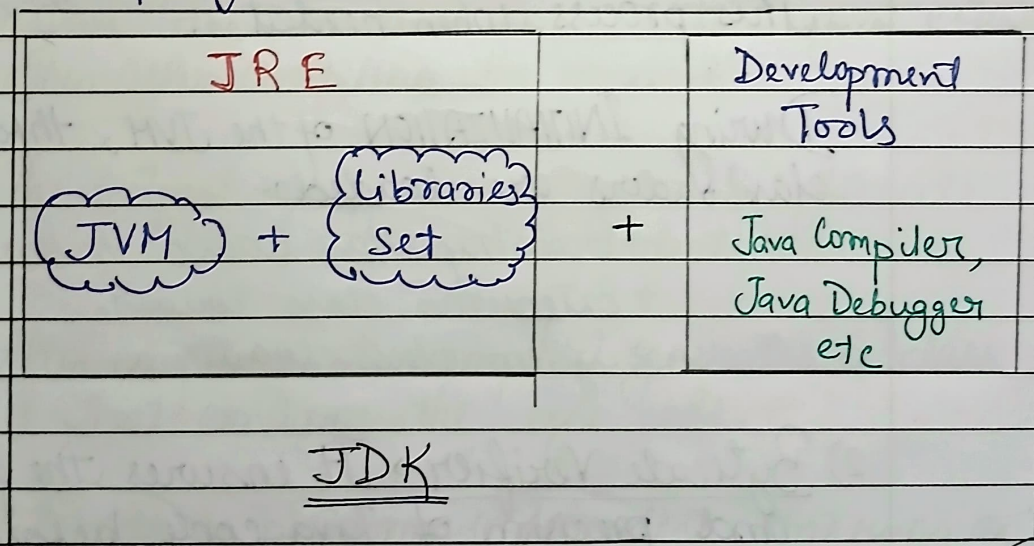
JRE handles → tasks such as :-

- loading classes.
- checking memory access.
- Accessing system resource

- _/_/_
- JRE acts as a software layer on top of the operating system to make Java programs platform-independent.

* Components of Java JRE

- ① Integration libraries include Java Database Connectivity (JDBC)
- ② Java Naming, Interface Definition Language (IDL)
- ③ Directory Interface ~~API~~ (JNDI).
- ④ Remote Method Invocation Over Internet Inter-Orb Protocol (RMI-IIOP).
- ⑤ Remote Method Invocation (RMI).
- ⑥ Scripting.



Working of JRE

- JDK & JRE both interact with each other to create a sustainable runtime environment that enables Java-based application to run seamlessly on any operating system.

JRE runtime architecture consists:-

- 1.) ClassLoader
- 2.) ByteCode verifier
- 3.) Interpreter.

1.) ClassLoader → Java ClassLoader dynamically loads all the classes necessary to run a Java program.

Because classes are only loaded into memory whenever they are needed, the JRE uses ClassLoader will automate this process when needed.

During INITIALIZATION of the JVM, three classLoaders are loaded:-

- Bootstrap class loader
- Extensions class loader
- System class loader

2.) Bytecode Verifier → It ensures the format and precision of Java code before passing it to the interpreter.

→ If the code violates system integrity or access rights, the class is considered corrupt & will not load.

(2) Interpreter → After loading the byte code successfully, the Java interpreter creates an object of the Java virtual machine that allows the Java program to run natively on the underlying machine.

- JRE has an object of JVM with it,
development tools → library classes.

Eg:- >>

```
class Nitin {  
    public static void main (String[] args) {  
        System.out.println ("Hi, Nitin");  
    }  
}
```

→ Save the above code with ".java" extension
Then after compiling the program.

→ The output of the Java compiler is byte code which is a platform-independent code.

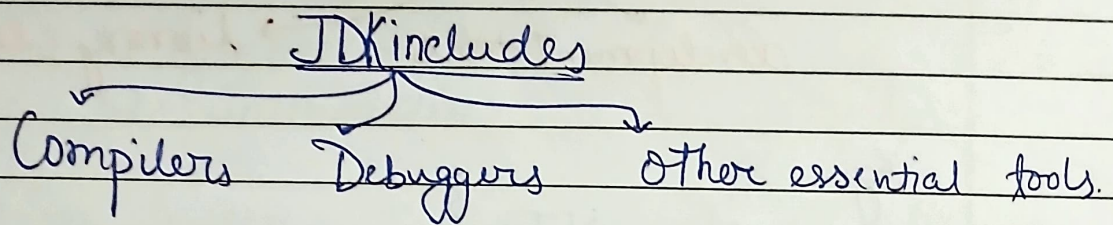
After compiling, the compiler generates a .class file that contains the byte code.

Bytecode is platform-independent that runs on all ~~over~~ devices which contain JRE

Java Development Kit (JDK)

→ Java development kit (JDK) is a cross-platform software development kit that provides tools & libraries needed to build Java-based applications and applets.

→ It works with JVM & JRE as a part of the core Java setup.



→ If we only want to run Java programs, installing JRE is enough.

→ BUT, if we want to ~~run~~ develop Java applications, we'll need JDK, which includes the JRE and additional development tools.

• JDK is a complete development kit for Java, it includes

- Java SE (Standard ^{Edition} ~~Enterprise~~)
- Java EE (Enterprise Edition)
- Java ME (Micro Edition)

* Contents of JDK

→ The JDK has a private JVM and a few other resources necessary for the development of a Java Application.

• JDK contains:-

→ JRE

→ An interpreter/loader

→ A compiler (javac)

→ An archiver (jar) & many more.

→ The Java Runtime Environment (JRE) in JDK is usually called private runtime because it is separated from the regular JRE and had extra content.

	<u>C</u>	<u>C++</u>	<u>Java</u>
1.)	Structured programming language	Object-oriented programming language.	Object-Oriented Programming Language
2.)	Compiled	Compiled	Compiled & interpreted
3.)	Middle-level	high level	high-level
4.)	Top-down	Top-down	bottom-up
5.)	32 keywords	60 keywords	50 keywords
6.)	platform-dependent	platform dependent	platform independent
7.)	not follows OOPs	follows OOPs	follows OOPs
8.)	'.c' extension	'.cpp' extension	'.Java' extension
9.)	generates '.exe' file	generates '.exe' file	generates '.class' file
10.)	Supports 'call by value' & 'call by reference' both	Supports 'call by value' & 'call by reference' both	Supports 'call by value' only.
11.)	It is not robust	It is robust	It is robust
12.)	Exception handling not present	Exception handling is present	Exception handling is present
13.)	There is no concept of threading	It supports the concept of threading	It supports the concept of threading.